



Programación Orientada a Objetos

De escribir instrucciones a modelar sistemas



CENTRO DE FORMACIÓN PROFESIONAL N°8
SMATA

CFP N°8
FORMACIÓN
PROFESIONAL
SMATA

Francisco Acuña

- ✓ Técnico Superior en Desarrollo de Software
- ✓ Instructor Certificado de Formación Profesional
- ✓ Especializaciones en Programación Web y Administración de Bases de Datos
- ✓ Google Gemini Educator Certified



Docente de programación, bases de datos e Inteligencia Artificial

Analista de Base de Datos



“ *Pasión y perseverancia son la verdadera arquitectura del éxito.
Y el éxito es vivir siendo felices con lo que hacemos...* ”



franciscoacuna.centro8@gmail.com



franacuna.tech

No empezamos de cero

Técnicas de Programación
sigue siendo la base:

variables

tipos de datos

operadores

condicionales

ciclos

funciones

arrays

eventos

PОО con Java
no borra lo
anterior: lo
vuelve más
explícito y más
organizado.

Cambia la pregunta principal

Antes:

¿Qué instrucciones
debe ejecutar el
programa?

Ahora agregamos:

¿Qué elementos
existen en el
problema?

¿Qué información
conoce cada uno?

¿Qué sabe hacer
cada uno?

¿Con quién
colabora?

¿Qué es un paradigma?

Un paradigma es una forma de pensar y organizar la solución de un problema.



En **programación estructurada** pensamos principalmente en instrucciones, funciones y datos.



En **Programación Orientada a Objetos** seguimos usando instrucciones, pero organizadas dentro de objetos que colaboran.

Java permite mezclar estilos, pero en este curso vamos a concentrarnos en orientación a objetos.

¿Qué es la Programación Orientada a Objetos?

Es una forma de construir software donde los objetos poseen **estado**, **características**, **comportamiento** e **identidad**, y colaboran para resolver un problema.

1. Estado: datos que conoce.

2. Características: propiedades o atributos que posee.

3. Comportamiento: acciones que puede realizar.

4. Identidad: ese objeto concreto dentro del sistema.

5. Responsabilidad: lo que le corresponde resolver.

Objeto = estado + características + comportamiento + identidad + responsabilidad

¿Por qué usamos objetos?

Usamos objetos para organizar sistemas que empiezan a crecer.



Ayudan a separar responsabilidades.



Ayudan a reducir el impacto de los cambios.



Ayudan a evitar código repetido cuando el diseño está bien pensado.



Ayudan a trabajar con piezas más claras.



Ayudan a conectar el modelo del programa con datos que luego podrán guardarse en una base de datos.



Pero la POO no garantiza buen diseño automáticamente. Un mal diseño orientado a objetos puede ser igual o peor que un programa desordenado.

¿Para qué se usa hoy en día POO con Java?



Clase no es lo mismo que objeto

La Clase / El Molde



Una clase define una estructura general.

Clase: Curso

Puede tener: nombre, cupo, cantidad de inscriptos

Puede hacer: hayVacantes(), inscribirAlumno()

Los Objetos / Las Instancias



Un objeto es un caso concreto creado a partir de esa clase.

Objetos concretos: "Programación Orientada a Objetos",
"Base de Datos", "Alfabetización IA"
Mismo molde, distintos objetos.

Estado, comportamiento y responsabilidad

Ejemplo: Curso

Estado:

nombre
cupo
cantidad de inscriptos

Comportamiento:

hayVacantes()
inscribirAlumno()

Responsabilidad:

- El Curso debería saber si tiene cupo disponible.
- Esa lógica no debería quedar repetida por toda la aplicación.
- Pregunta clave del diseño: ¿Quién debería hacerse cargo de esto?

Los objetos colaboran

Una aplicación no es una clase gigante ni una lista interminable de instrucciones.



El sistema aparece por la colaboración entre objetos.

Conceptos que iremos descubriendo

Fundamentos de POO que descubriremos paso a paso:



Abstracción:
quedarse con lo
relevante del
problema.



Encapsulamiento:
proteger decisiones
internas y exponer
una forma
controlada de uso.



Relaciones:
objetos que
conocen,
contienen o usan
otros objetos.



Herencia:
especializar tipos
cuando realmente
tiene sentido.



Polimorfismo:
misma abstracción,
distintos
comportamientos
posibles.